

Joe Pan — Senior iOS Engineer

shinren.pan@gmail.com github.com/shinrenpan shinrenpan.github.io

SUMMARY

Over **15 years** of hands-on iOS development experience across a wide range of app types — MFi hardware accessories, instant messaging, video streaming, and fintech — participating in every stage of the platform's evolution from pure Objective-C to modern Swift and SwiftUI.

Focused on **architecture design**, with a core conviction that architecture should be grounded in Apple's own frameworks — studying the design intent behind UIKit, SwiftUI, and Swift Concurrency, then building loosely coupled, maintainable solutions from that foundation rather than adopting third-party frameworks that diverge from the platform's native model. This **native-first** philosophy extends to dependency management: Apple's native APIs are always the first choice, keeping maintenance overhead and binary size in check. Adapts quickly to existing team conventions and designs custom solutions when established patterns fall short; MVVMC is a direct expression of this conviction.

Years of hands-on experience across nearly every common app feature type means problems are diagnosed fast — most scenarios have been encountered before, making it possible to zero in on the key information and reach a viable solution quickly.

SKILLS

Languages

Swift 3.0 – present Objective-C 2009 – present

UI

Proficient in both UIKit and SwiftUI, capable of building complex custom interfaces from scratch. The two are combined as the core foundation of the MVVMC architecture, with `UIHostingController` as the navigation unit and SwiftUI handling rendering. Early study of cocos2d for iPhone established a solid 2D graphics foundation, which carried over into deep familiarity with SpriteKit — used as a fallback for animations and interactions that UIKit and SwiftUI cannot achieve natively.

UIKit SwiftUI UIHostingController hybrid AutoLayout Core Animation SpriteKit

Architecture

Not tied to a single pattern — prioritises maintainability and team adaptability. Has led or contributed to the adoption and refactoring of multiple architectures; designs custom solutions when standard ones prove inadequate.

MVC MVVM Clean Swift TCA MVVMC (custom) @Observable / @MainActor

Swift Concurrency

Migrated an existing Swift 5+ codebase to full Swift 6 readiness through systematic adoption of Swift Concurrency.

async/await @MainActor actor Sendable Task / TaskGroup

Networking

URLSession is the default (native-first); gRPC is used where the project requires it. Practical experience with Alamofire and other third-party networking libraries.

URLSession gRPC (grpc-swift / connect-swift) Alamofire

Real-Time Communication

Hands-on production experience with all major messaging protocols, covering both text-based communication and live video streaming.

WebSocket XMPP MQTT SignalR VoIP Live Streaming (HaishinKit / jkplayer / ReplayKit)

Persistence

Native solutions are preferred; major third-party options have also been used in production.

CoreData SwiftData Keychain NSKeyedArchiver Realm fmdb

Bluetooth

CoreBluetooth iBeacon

Healthcare Integration

Proficient in FHIR resource reading and writing, SMART on FHIR (OAuth 2.0 + PKCE), TW Core IG (R4), and Apple FHIRModels; familiar with the real-world Taiwan healthcare IT landscape (SMART on FHIR vs. static Bearer Token trade-offs).

FHIR SMART on FHIR TW Core IG HAPI FHIR Apple FHIRModels TWCoreFHIRModels ASWebAuthenticationSession

Toolchain

Category	Tools
Version Control	Git
Package Management	Swift Package Manager, CocoaPods, Carthage
Project Management	XcodeGen, Tuist
CI / CD	Xcode Cloud, Jenkins, Fastlane, Firebase App Distribution
AI Development	Claude (CLAUDE.md + Skill specs for systematic code generation)

EXPERIENCE

DRACO EVOLUTION

Apr 2025 – Jun 2026

Senior iOS Engineer · Took over from an outsourced team; sole iOS engineer at the company

Independently assessed code quality, paid down technical debt, and led the technical architecture evolution.

- TCA upgrade and removal:** Inherited the project at TCA 1.15.2 and upgraded incrementally to 1.22.2 (1.18.0 → 1.19.1 → 1.20.2 → 1.21.1 → 1.22.1 → 1.22.2). During routine maintenance and feature development, identified that TCA's built-in Navigation Stack could not accommodate the app's non-standard routing requirements, prompting a gradual architectural rewrite that went live on 18 Sep 2025.
- Custom MVVMC architecture (GitHub):** After removing TCA, found that SwiftUI's native NavigationStack still couldn't handle complex non-standard routing (e.g. simultaneously dismissing a checkout flow, switching tabs, and clearing a navigation stack). Combined with a deliberate decision not to base the entire codebase on a third-party architecture framework, drew on concepts from TCA and MVI to design MVVMC from scratch in native Swift — extending MVVM with a HostController layer (C): M handles State / Domain Models / DTOs; VM uses `@Observable @MainActor` with a single entry point (`doAction`) for business logic; V is pure SwiftUI with zero navigation or business logic; C (`UIHostingController`) is the sole Router with all navigation centralised through `AppRouter.shared`. Open-sourced with an MCP Server so Claude Code can access the architecture spec from any project.
- Swift 5 → Ready for Swift 6 migration:** Inherited a Swift 5+ codebase and systematically introduced `async/await`, `@MainActor`, `actor`, and `Sendable`, completing the Swift 6 compatibility migration in parallel with the TCA removal and MVVMC refactor.
- Xcode Cloud CI/CD pipeline:** Dev builds distribute automatically to TestFlight internal testers; production builds distribute to TestFlight external testers — fully automated, no manual steps required.
- US App Store release:** Published exclusively on the US App Store to comply with SEC regulations; subsequently taken down following an API shutdown.
- AI-assisted development workflow (Claude + Skill):** Authored `CLAUDE.md` and a suite of Skill documents (SwiftUI, ViewModel, HostController, Model, Swift Concurrency, etc.) so that Claude generates feature code that conforms to the project's existing architecture, significantly reducing time-to-feature.

Evervictory Technology

Sep 2023 – Sep 2024

iOS Engineer · Lead developer in a 3-person iOS team; the other two focused on an in-house K-Line library and a separate VPN app

Primary maintainer of a fintech trading platform and [JinTian GT](#) (a precious-metals trading app), plus contributions to an in-house IM client built on Tinode.

Fintech Trading App (demo)

- Introduced XcodeGen to eliminate persistent `.xcodeproj` merge conflicts in a multi-developer workflow.
- Integrated SignalR for real-time quote streaming; resolved UICollectionView cell-update jank during scroll using `CADisplayLink + RunLoop`.
- Replaced UITableView with UICollectionView across the codebase to enable richer layouts and more flexible cell composition; maintained and extended the in-house K-Line charting library.
- Wrote shell scripts to switch between dev / QAT / production environments; implemented in-app live language switching.

JinTian GT App (demo)

- Swift / Objective-C mixed-language development; integrated gRPC for data transport.
- Applied the [Rate Limiting UITableView/UICollectionView Reloads](#) technique to throttle excessive reloads triggered by high-frequency gRPC updates.
- Implemented a review-mode UI-hiding mechanism to pass App Store review.
- Built a CI/CD pipeline with Jenkins + Fastlane; used Firebase App Distribution for dev / QAT builds.
- Adopted Xcode 15 asset symbol generation; built a JavaScript bridge for bidirectional HTML5 ↔ native communication.

IM Client (Tinode-based)

- Removed Storyboard in favour of fully programmatic UI to enable deep customisation of Tinode's default interface.
- With no Golang engineer on the team, self-taught Go and read through the Tinode server codebase to understand server behaviour and accurately define the boundaries of IM feature development.

Heng Yuan Technology Co., Ltd.

Apr 2022 – Jul 2023

iOS Engineer · Sole iOS engineer on the project (demo)

Took over an Objective-C codebase and led the technical modernisation of an app covering short video, long video, comics, novels, games, live streaming, and chat.

- Led ObjC → Swift migration:** Defined a gradual strategy — all new features written in Swift, legacy modules refactored incrementally.
- Introduced XcodeGen to eliminate Git project-file conflicts; researched Tuist and evaluated both tools.
- Refactored local HLS playback architecture:** The original approach downloaded m3u8 files locally and served them through GCDWebServer — high maintenance cost. Replaced with a custom `AVAssetResourceLoaderDelegate`; substitutes `file://` with a custom `local://` scheme so AVPlayer delegates the request to the loader, which swaps back to `file://` and reads the local data. Removed the GCDWebServer dependency and reduced binary size ([reference article](#)).
- Built comic reader, novel reader, and embedded game pages — all using WebView with bidirectional native communication.
- Integrated a live-streaming SDK written in Flutter, resolving Flutter module embedding issues and debugging across framework boundaries.
- Coordinated with a third-party signing vendor for enterprise distribution outside the App Store.

Gamania Group

Jun 2019 – Feb 2022

iOS Contract Engineer

BeanFun App

- Collaborated with the team to introduce Clean Swift architecture, progressively migrating Objective-C modules to Swift.
- BeanFun loads the majority of its features in WebViews and requires hot-reload support; implemented deep WebView ↔ native bidirectional communication covering JavaScript bridge design, event propagation, and page state synchronisation.

Gamania Group

Sep 2018 – Mar 2019

iOS Contract Engineer · Lead iOS developer

In-House Enterprise App "teamup!"

Internal project management and approval platform supporting instant messaging, digital sign-off, and access control. Used by Gamania's CEO during an Antarctic expedition as a remote management tool ([press coverage](#)).

- Optimised for low-bandwidth conditions: cached API responses with fmdb for offline use; implemented multi-tier image caching (thumbnail-first display, background download and swap to full resolution) to keep the app functional on slow connections.

Wistron ITS

Aug 2016 – Mar 2018

iOS Engineer · On-site at Cathay Life Insurance

Primary maintainer of the [Cathay Life Insurance](#) app. The codebase was Objective-C; following Cathay's internal decision to adopt Swift, took responsibility for implementing new features in Swift and progressively migrating legacy Objective-C modules.

Hoter Information

Mar 2016 – Jun 2016

iOS Engineer

Developed an in-house hotel workforce management app for staff scheduling, real-time reporting, and check-in. The backend chose SignalR for the chat feature — an uncommon choice at a time when XMPP and MQTT dominated the space — providing first-hand experience with the SignalR protocol. Also implemented geofenced check-in using Core Location.

Internet Mobile Technology

Apr 2015 – Jan 2016

iOS Engineer

Built a shopping app centred on a gamification concept inspired by Tap Titans: users earn virtual currency or discount coupons through in-app gameplay, redeemable in the store. Implemented chat using XMPP and built the game module with SpriteKit — the first experience integrating a game engine into an e-commerce app.

Earlier Experience (2010 – 2015)

Company	Period	Key Work
PiPiMy	Jan 2015 – Mar 2015	C2C second-hand trading app (Beta)
Timeline Technology	May 2012 – Jun 2014	Template-based storefront app framework / coupon app
JamZoo	Nov 2012 – Jun 2013	Migrated "Single Bank" app chat from timer polling to MQTT (following Facebook's adoption of MQTT), resolving lag; built offline-capable car rental app and HTML5 e-book WebView
Viamedia Mobile Corporation	Sep 2010 – Feb 2012	Media Player interactive app , mini-games, BLE
Sunplus Semiconductor	Mar 2010 – Jul 2010	MFi hardware accessory app , internet radio, local music playback

PROJECTS

FHIRpass

GitHub

A universal medical identity iOS App MVP built on a three-track architecture (offline QR registration, SMART on FHIR authorization, iPad scan-to-FHIR check-in), addressing the two biggest barriers to third-party healthcare integration: hospital firewall resistance and personal data compliance. Full-stack solo development: iOS (SwiftUI + SwiftData), Python FastAPI, Docker (HAPI FHIR + SMART Launcher), and a web-based scanning frontend.

- Track 1 (Offline QR):** Compact string format (UTF-8 → zlib → Base64); local encrypted storage via SwiftData; middleware has zero contact with personal data
- Track 2 (SMART on FHIR):** `ASWebAuthenticationSession` + OAuth2 + PKCE; token locked to Keychain with `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly`
- Track 3 (iPad Counter):** FastAPI + `html5-qrcode`; direct `POST /Patient` to HAPI FHIR with TW Core IG-compliant resources; full appointment lifecycle proposed → booked

MVVMC

GitHub

A four-layer iOS architecture (M / VM / V / C) designed to overcome the limitations of SwiftUI's native navigation. Uses `UIHostingController` as the navigation unit and centralises all routing through `AppRouter.shared`, keeping SwiftUI views free of any navigation dependencies. Includes a runnable Demo project and an MCP Server for Claude Code integration.

TWCoreFHIRModels

GitHub

A Swift Package providing strongly typed TW Core IG extensions on top of Apple's FHIRModels. Adds a `.twCore` namespace API so developers can set Taiwan-specific fields directly and call `validateTWCore()` to verify SHALL-required fields automatically — no hardcoded Profile or CodeSystem URLs needed.

- Strongly typed `.twCore` namespace — assign ID card number, declare Profile, validate SHALL fields without any hardcoded URLs
- Compliance strategy aligned to FHIR RFC 2119 (SHALL / SHOULD / MAY) with validation granularity matched to spec
- Swift 6.2 strict concurrency, GitHub Actions CI, distributed via SPM

WebParser

GitHub

A Swift Package for web parsing built on WKWebView off-screen rendering, designed for dynamic pages that require JavaScript execution (SPAs, dynamic DOM). Exposes a type-safe generic Mapper interface (JSON → `Decodable`, custom Regex extraction), fully Swift 6 strict concurrency compliant, tested with Swift Testing, distributed via SPM with GitHub Actions CI.

EDUCATION

Institution	Programme	Period
Chaoyang University of Technology	Information Management (evening programme)	Sep 2004 – Jan 2007
Kao Yuan University (5-year)	Electronic Engineering	—
Institute for Information Industry	iOS Development (6-month programme)	2009

Graduation project: developed rhythm game *Block Ten* using cocos2d for iPhone, winning *Gold Award at the 2009 Digital Content Creation Competition (Mobile Game category)*. After graduating, contracted with classmates to build a [music app](#).

Languages: Mandarin Chinese (native) English (technical reading and writing)